

DUSPAR: DUAL-STATE SPARSIFYING RECURRENT UNIT WITH FEEDBACK MODULATION FOR COMPUTE-EFFICIENT SPEECH PROCESSING

Zixiao Li, Sheng Zhou, Longbiao Cheng, Shih-Chii Liu

Institute of Neuroinformatics, University of Zurich and ETH Zurich, Zurich, Switzerland

ABSTRACT

We introduce the Dual-state Sparsifying Recurrent Unit (DuSpaR) as a computationally efficient building block for speech processing models on resource-constrained edge devices. It employs dual-state recurrence to modulate its input vectors in a stateful feedback loop. Its recurrent cells sparsify the input vector operand involved in matrix-vector multiplication using ReLU activation. By skipping the zero entries dynamically, inference-time savings in multiply-accumulate operations and weight memory fetches can be achieved. We evaluate DuSpaR on three speech tasks: keyword spotting (KWS) on the Google Speech Commands dataset, spoken language understanding (SLU) on the Fluent Speech Commands dataset, and speech enhancement (SE) on the Voice Bank + Demand (VBD) dataset. At similar parameter counts, DuSpaR requires 51.0% and 68.4% less computation than Gated Recurrent Unit (GRU) on KWS and SLU, respectively, while achieving higher accuracy, and 50.1% less computation on SE while maintaining similar quality. At comparable computational cost and across a range of model sizes, DuSpaR also achieves higher KWS/SLU accuracy and better SE quality than other sparsity-aware recurrent models. Ablation studies show that compared with the single-state recurrence baseline, dual-state recurrence reduces the effective compute by $3.1\times$ to $11.3\times$ at similar task performance.

Index Terms— Dual-State Sparsifying Recurrent Unit, Computationally Efficient Networks, Speech Enhancement, Keyword Spotting, Spoken Language Understanding.

1. INTRODUCTION

Neural networks for speech processing are widely employed in mobile platforms including smartphones, smart glasses, wearables, and hearables. They support voice interaction and improve speech intelligibility, performing tasks such as keyword spotting (KWS), automatic speech recognition (ASR), spoken language understanding (SLU), and speech enhancement (SE). Compared with remote processing in the cloud, deploying these networks locally on the device offers lower latency, higher reliability, and better user privacy [1]. However, the limited compute and memory resources available on device create a significant challenge for their deployment [2], necessitating the design of compact and efficient networks.

Recurrent neural networks (RNNs) such as Gated Recurrent Unit (GRU) [3] and Long Short-Term Memory (LSTM) [4] are well suited to streaming speech processing because they encode the input context as a fixed-size recurrent state, whose storage does not grow with the duration of the input stream [5, 6]. New recurrent models such as LRU [7], xLSTM [8], S4 [9], and Mamba [10] show

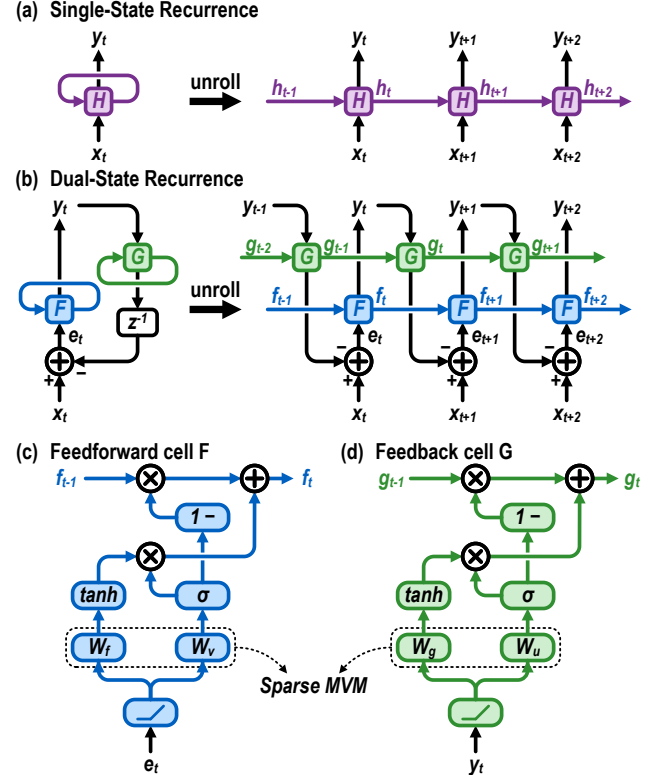


Fig. 1. Illustration of the proposed Dual-state Sparsifying Recurrent Unit (DuSpaR). (a) Conventional single-state recurrence. (b) Proposed dual-state recurrence adopted in DuSpaR. (c) Implementation of the feedforward recurrent cell F . (d) Implementation of the feedback recurrent cell G . All vector operands in matrix-vector multiplication (MVM) are sparsified by ReLU.

continual improvement in performance and training efficiency for sequence modeling. Nevertheless, these recurrent layers perform dense matrix-vector multiplication (MVM) at every inference step, leading to a large number of multiply-accumulate (MAC) operations and weight memory fetches. Therefore, reducing the inference cost of recurrent layers without compromising task performance is an important target for on-device speech-processing networks.

Several methods can be employed to optimize the inference cost of RNNs. Quantization [11] of weights and activations reduces the unit cost per MAC. Pruning [12] provides static sparsification of weights, allowing zero-weight MACs to be skipped. Dynamic computation [6, 13–16] instead relies on dynamic sparsification of activations, which enables input-dependent skipping of MACs and memory fetches during inference. These models have demonstrated significant throughput improvement and energy reduction when

deployed on custom hardware [17, 18].

In this work, we introduce Dual-state Sparsifying Recurrent Unit (DuSpaR), a novel recurrent building block (Fig. 1) improving the trade-off between task performance and compute cost compared to previous dynamic computation methods. Our contributions are as follows:

1. A dual-state recurrence scheme (Section 2.1) that modulates the input temporal sequence through a stateful feedback loop. Ablation studies (Section 4) show that the feedback reduces the number of MACs at similar task performance compared to conventional single-state recurrence.
2. A sparsifying recurrent cell (Section 2.2) that uses ReLU to sparsify the vector operand in MVM while maintaining a bounded state using hyperbolic tangent activation. Coupling two such cells through dual-state recurrence results in the proposed DuSpaR.
3. Comprehensive evaluation of DuSpaR on multiple speech tasks (Section 3), covering both classification and regression. Comparisons across model sizes show that DuSpaR outperforms both dense and sparse recurrent models, achieving better task performance at comparable computational cost.

2. METHODS

In the following subsections, activation and state vectors are denoted by lowercase bold letters such as $\mathbf{x}$ and $\mathbf{y}$, weight matrices by uppercase bold letter $\mathbf{W}$, and bias vectors by lowercase bold letter $\mathbf{b}$. Nonlinear functions such as $\tanh$ and ReLU are applied element-wise, and $\odot$ denotes elementwise product of two vectors.

2.1. Dual-State Recurrence

Fig. 1 (a) illustrates a conventional single-state recurrent layer unrolled over time, with inputs $[\mathbf{x}_t]_{t=1}^T$, outputs $[\mathbf{y}_t]_{t=1}^T$, and recurrent states $[\mathbf{h}_t]_{t=1}^T$. At each time step t , the recurrent cell $\mathbf{H}$ computes the new state $\mathbf{h}_t$ from the current input $\mathbf{x}_t$ and the previous state $\mathbf{h}_{t-1}$ as $\mathbf{h}_t = \mathbf{H}(\mathbf{x}_t, \mathbf{h}_{t-1})$. In a multi-layer network, multiple recurrent cells are stacked in a feedforward manner. Therefore, information flows *one-way* from the inputs $[\mathbf{x}_t]_{t=1}^T$ to the outputs $[\mathbf{y}_t]_{t=1}^T$.

Similar to models [19] inspired by the prevalent local recurrent processing in brains [20], we introduce the dual-state recurrence shown in Fig. 1 (b). The two recurrent cells $\mathbf{F}$ and $\mathbf{G}$ are coupled in a feedback loop, with separate recurrent states $[\mathbf{f}_t]_{t=1}^T$ and $[\mathbf{g}_t]_{t=1}^T$. $\mathbf{F}$ sends *feedforward* information from $[\mathbf{e}_t]_{t=1}^T$ to $[\mathbf{y}_t]_{t=1}^T$, while $\mathbf{G}$ sends *feedback* information from $[\mathbf{y}_t]_{t=1}^T$ to $[\mathbf{e}_t]_{t=1}^T$. $\mathbf{e}_t$ is computed by subtracting the previous feedback state $\mathbf{g}_{t-1}$ from the input $\mathbf{x}_t$. In other words, the input sequence $[\mathbf{x}_t]_{t=1}^T$ is *modulated* by the feedback states $[\mathbf{g}_t]_{t=1}^T$. We hypothesize that this stateful feedback modulation of the inputs enhances the expressiveness of the model, and empirically confirm its effectiveness through an ablation study (Section 4). Mathematically, the dual-state recurrence is described by Eqs. 1–4, where $\mathbf{b}_g$ and $\mathbf{b}_f$ are trainable bias vectors.

$$\mathbf{e}_t = \mathbf{x}_t - (\mathbf{g}_{t-1} + \mathbf{b}_g) \quad \triangleright \text{Feedforward cell input} \quad (1)$$

$$\mathbf{f}_t = \mathbf{F}(\mathbf{e}_t, \mathbf{f}_{t-1}) \quad \triangleright \text{Feedforward cell state update} \quad (2)$$

$$\mathbf{y}_t = \mathbf{f}_t + \mathbf{b}_f \quad \triangleright \text{Feedback cell input} \quad (3)$$

$$\mathbf{g}_t = \mathbf{G}(\mathbf{y}_t, \mathbf{g}_{t-1}) \quad \triangleright \text{Feedback cell state update} \quad (4)$$

2.2. Sparsifying Recurrent Cell

To dynamically reduce the MAC operations and weight memory fetches, we employ the sparsifying recurrent cell shown in Fig. 1 (c) for $\mathbf{F}$ and Fig. 1 (d) for $\mathbf{G}$. The cell is adapted from minGRU [21, 22], which involves two MVMs, one for computing the update gate, and the other for the update candidate. We sparsify both MVMs by using ReLU to zero out the negative elements of the input vector. In addition, we apply $\tanh$ activation to the update candidate to ensure that the dense state vector stays bounded even for long sequences. The sparsifying recurrent cell for $\mathbf{F}$ is given by Eqs. 5–8, where σ is the sigmoid activation and the differences with minGRU are highlighted in purple.

$$\mathbf{e}_t^+ = \text{ReLU}(\mathbf{e}_t) \quad \triangleright \text{Input sparsification} \quad (5)$$

$$\mathbf{v}_t = \sigma(\mathbf{W}_v \mathbf{e}_t^+ + \mathbf{b}_v) \quad \triangleright \text{Update gate} \quad (6)$$

$$\tilde{\mathbf{f}}_t = \tanh(\mathbf{W}_f \mathbf{e}_t^+) \quad \triangleright \text{Update candidate} \quad (7)$$

$$\mathbf{f}_t = (1 - \mathbf{v}_t) \odot \mathbf{f}_{t-1} + \mathbf{v}_t \odot \tilde{\mathbf{f}}_t \quad \triangleright \text{Gated state update} \quad (8)$$

Similarly, the sparsifying recurrent cell for $\mathbf{G}$ is given by Eqs. 9–12.

$$\mathbf{y}_t^+ = \text{ReLU}(\mathbf{y}_t) \quad \triangleright \text{Input sparsification} \quad (9)$$

$$\mathbf{u}_t = \sigma(\mathbf{W}_u \mathbf{y}_t^+ + \mathbf{b}_u) \quad \triangleright \text{Update gate} \quad (10)$$

$$\tilde{\mathbf{g}}_t = \tanh(\mathbf{W}_g \mathbf{y}_t^+) \quad \triangleright \text{Update candidate} \quad (11)$$

$$\mathbf{g}_t = (1 - \mathbf{u}_t) \odot \mathbf{g}_{t-1} + \mathbf{u}_t \odot \tilde{\mathbf{g}}_t \quad \triangleright \text{Gated state update} \quad (12)$$

Our sparsifying recurrent cell is based on minGRU because its recurrent states are only involved in elementwise operations instead of MVM as in GRU or LSTM. This allows us to sparsify the MVMs, which dominate the overall computational cost, without obscuring the information flow through time by keeping the recurrent states as dense vectors.

For a DuSpaR layer with input dimension N and output dimension M , there are $4NM + 2(N + M)$ trainable parameters in total, including $\mathbf{W}_f, \mathbf{W}_g, \mathbf{W}_u, \mathbf{W}_v, \mathbf{b}_f, \mathbf{b}_g, \mathbf{b}_u, \mathbf{b}_v$. With elementwise operations for the gated state updates (Eqs. 8 and 12) included, the number of effective MACs per time step is reduced from $4NM + 2(N + M)$ to $2(o_e + o_y)NM + 2(N + M)$, where o_e and o_y are the average occupancies of $\mathbf{e}_t^+$ (Eq. 5) and $\mathbf{y}_t^+$ (Eq. 9), respectively. The MAC reduction ratio is approximately $(o_e + o_y)/2$. Likewise, the weight memory fetches per time step are reduced from $4NM$ to $2(o_e + o_y)NM$, by a ratio of $(o_e + o_y)/2$.

2.3. Related Work

Efficient RNN The fine-grained activation sparsity in DuSpaR is complementary to other techniques for reducing RNN inference cost, including quantization [11], pruning [12], and coarse-grained frame skipping [15]. Combining these techniques further reduces the inference cost of speech processing RNNs [23].

Activation Sparsity Several prior works [6, 13, 14] also exploit fine-grained activation sparsity for efficient recurrent inference. Delta Network [13] skips entries of the input and state vectors whose activation change across two time steps is below a set threshold. EGRU [14] introduces a stateful gating mechanism with a trainable threshold for sparse, event-based updates. Dynamic Gated RNN [6] skips the update of a fixed fraction of recurrent neurons based on their update-gate magnitudes. These methods introduce sparsity by modifying or approximating an underlying dense model such as GRU. In contrast, our proposed DuSpaR is intrinsically sparse and achieves better trade-offs between task performance and effective MAC operations at comparable model sizes (Section 4).

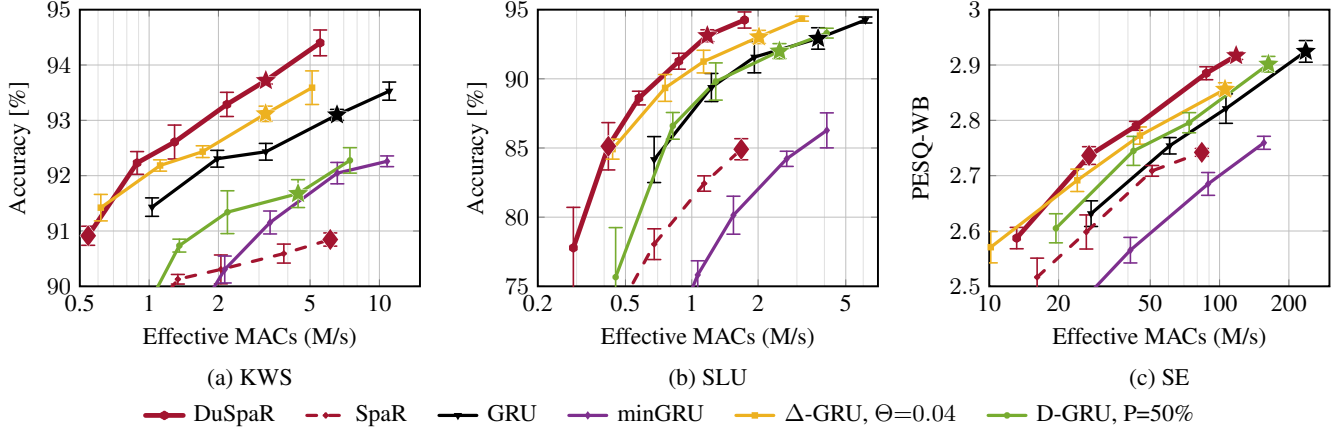


Fig. 2. Task performance versus total effective computation for (a) KWS, (b) SLU, and (c) SE. Error bars indicate standard deviation over five runs. Each curve shows results for one recurrent model at the layer widths described in Section 3.2. Stars (★) mark the DuSpaR, GRU, Δ -GRU, and D-GRU configurations in Table 1, which have similar parameter counts within each task. Diamonds (◆) mark the DuSpaR–SpaR pairs used to evaluate the effectiveness of dual-state recurrence by comparing computational cost at similar task performance.

ReLU RNN The ReLU activation in DuSpaR has been used in other recurrent models. IRNN [24] produces sparse recurrent states using the ReLU nonlinearity, while DuSpaR retains dense recurrent states and applies the sparsification only to the recurrent cell inputs. Li-GRU [25] simplifies the GRU architecture by removing the reset gate and replacing the tanh activation with ReLU in calculating the update candidate. However, the update candidate is only involved in elementwise operations, so the computational savings are negligible.

3. EXPERIMENTAL SETUP

In Section 3.1, we evaluate DuSpaR on three speech tasks with different objectives and temporal context windows: KWS, SLU, and SE. For each task, we use a specific GRU-based architecture and conduct comparison experiments by replacing its GRU modules with DuSpaR and other recurrent models described in Section 3.2.

3.1. Tasks

Keyword Spotting KWS detects target keywords using word-level context spanning hundreds of milliseconds. We use the Google Speech Commands Dataset (GSCD) [26], which contains 105,000 one-second utterances covering 35 keyword classes. All recordings from GSCD are sampled at 16 kHz and processed using 32-ms Hann windows with a 16-ms hop. Each frame’s spectrogram is then computed using a 512-point Fast Fourier Transform (FFT) and mapped to 64 mel bands spanning from 50 Hz to 8 kHz. The mel-band features are log-transformed and standardized using training-set statistics. The KWS model uses two GRU layers and one fully connected (FC) classification layer. We train the model for up to 100 epochs with batch size 64 and standard cross-entropy (CE) loss.

Spoken Language Understanding SLU infers user intent at the sentence level from a context window spanning several seconds. We use the Fluent Speech Commands Dataset (FSCD) [27], which contains 19 hours of English smart-home commands recordings from 97 speakers, labeled with 31 intents defined by action-object-location combinations. We use the identical KWS audio front-end for SLU, and a model with two GRU layers and one FC layer [18]. To reduce computation, two temporal pooling layers, each with a window size of 4, are applied to the GRU outputs. We train the model for 200

epochs with batch size 64 and the Connectionist Temporal Classification (CTC) loss [28]. During the first 100 epochs, we supplement the CTC loss with an auxiliary CE loss to improve convergence.

Speech Enhancement SE estimates clean speech from noisy inputs lasting up to tens of seconds. We use the Voice Bank + DEMAND (VBD) dataset [29]. The training set mixes clean speech from 28 of the 30 speakers in the Voice Bank dataset [30] with 10 of the 15 noise types in the DEMAND dataset [31] at four signal-to-noise ratio (SNR) levels: 0, 5, 10, and 15 dB. For testing, clean speech from the remaining two speakers is mixed with the remaining five noise types at SNRs of 2.5, 7.5, 12.5, and 17.5 dB. To target 2 ms ultra-low enhancement latency [32], we sample audio at 16 kHz and use 32-sample (2 ms) frames with a 16-sample (1 ms) hop.

The SE model directly processes these time-domain frames using an encoder-mask-decoder architecture [32]. A linear encoder maps each frame to learned features. Four GRU layers followed by one FC layer estimate a mask that multiplies the encoder features element-wise. A linear decoder converts the masked features to waveform frames for audio reconstruction by Hann-windowed Overlap-and-Add (OLA). We train for 280 epochs with batch size 32, and only the mean-squared error (MSE) loss is used for the first 200 epochs. For the last 80 epochs, we adopt the loss function and weighting factors from [32]. This loss function combines MSE, perceptual-based loss terms from [33] and [34], and a negative Scale-Invariant Signal-to-Noise Ratio (SI-SNR) loss term. We evaluate SE performance using PESQ-WB [35], STOI [36], and scale-invariant signal-to-distortion ratio (SI-SDR) [37].

3.2. Recurrent Model Comparison and Ablation

We compare DuSpaR with both dense and sparsity-aware recurrent models to evaluate their trade-offs between task performance and computational cost. For each network described in Section 3.1, we replace its GRU layers with DuSpaR or other recurrent models listed below while keeping the remaining network structure fixed. For the dense variants, apart from the default GRU layer, we include minGRU [21, 22] due to its connection with DuSpaR (Section 2.2). For the sparse variants, we include Δ -GRU as a representative of Delta Networks [13] and D-GRU as a representative of Dynamic Gated RNN [6]. In addition, as an ablation study on the contribution of the dual-state recurrence (Section 2.1), we also consider a variant of

Table 1. KWS and SLU accuracy, SE metrics, and computational cost of recurrent models with comparable parameter counts within each task. Entries with $\pm$ report the mean and standard deviation over five runs. Dashes indicate entries that are not applicable.

Task	Model	Para. (K)	Layer Width	Dense MACs (M/s) ¹	Effective MACs (M/s) ² ↓	Occ. ³ ↓ (%)	Accuracy ↑ (%)	PESQ-WB ↑	STOI ↑ (%)	SISDR ↑ (dB)
KWS	GRU	105.9	96	6.55	6.55	100	93.10 $\pm$ 0.10	-	-	-
	Δ -GRU ⁴	105.9	96	6.55	3.21	49.0	93.12 $\pm$ 0.14	-	-	-
	D-GRU ⁵	105.9	96	6.55	4.43	67.7	91.68 $\pm$ 0.25	-	-	-
	DuSpaR	103.7	128	6.42	3.21	50.0	93.72 $\pm$ 0.06	-	-	-
SLU	GRU	105.6	96	3.74	3.74	100	92.92 $\pm$ 0.77	-	-	-
	Δ -GRU ⁴	105.6	96	3.74	2.02	53.8	93.03 $\pm$ 0.48	-	-	-
	D-GRU ⁵	105.6	96	3.74	2.50	66.8	92.00 $\pm$ 0.54	-	-	-
	DuSpaR	103.3	128	3.08	1.18	38.3	93.12 $\pm$ 0.43	-	-	-
SE	noisy	-	-	-	-	-	-	1.97	92.10	8.45
	GRU	239.5	96	236.78	236.78	100	-	2.92 $\pm$ 0.02	94.02 $\pm$ 0.05	18.43 $\pm$ 0.05
	Δ -GRU ⁴	239.5	96	236.78	105.58	44.6	-	2.86 $\pm$ 0.01	93.76 $\pm$ 0.02	18.34 $\pm$ 0.05
	D-GRU ⁵	239.5	96	236.78	162.98	68.8	-	2.90 $\pm$ 0.01	93.91 $\pm$ 0.09	18.31 $\pm$ 0.06
	DuSpaR	238.8	116	236.41	118.04	49.9	-	2.92 $\pm$ 0.01	93.82 $\pm$ 0.13	18.34 $\pm$ 0.03

¹ All-layer MAC cost without sparsity-based skipping; M/s denotes millions of MACs per second.

² All-layer MAC cost after accounting for sparsity-based skipping.

³ Occ. is shorthand for occupancy, where occupancy = (Effective MACs/Dense MACs) $\times$ 100%.

⁴ Δ -GRU uses $\Theta = 0.04$ for all tasks. ⁵ D-GRU uses $P = 50\%$ for all tasks.

DuSpaR without the feedback path. We call this variant Sparsifying Recurrent Unit (SpaR) and it is equivalent to retaining only the feed-forward path of DuSpaR following Eq. 5-8 with $e_t = x_t$. We also experimented with IRNN [24] and Li-GRU [25], but exploding gradients prevented successful training in our SLU and SE experiments. We therefore excluded both models from our final comparison.

We note that Δ -GRU and D-GRU use the delta threshold Θ and update ratio P , respectively, to control the trade-off between task performance and computational cost. Suitable values for these hyperparameters depend on the task, model, and desired trade-off. We sweep $\Theta \in \{0.02, 0.04, 0.06\}$ for Δ -GRU and $P \in \{25\%, 50\%\}$ for D-GRU, and select $\Theta = 0.04$ and $P = 50\%$ because they provide favorable trade-offs across all tasks. Although other values may yield better trade-offs for specific task and model configurations, identifying them can require substantial tuning effort, representing a potential limitation of both algorithms.

We define the layer width as the output dimension of each recurrent layer. For DuSpaR, it is the dimension of the feedforward state f_t in Eq. 2. For KWS and SLU, we use a common layer-width grid of $\{32, 48, 64, 96, 128\}$, extended to 192 for DuSpaR and to $\{192, 256\}$ for SpaR and minGRU. For SE, the common grid is $\{32, 48, 64, 96\}$, with additional widths of 116 for DuSpaR and 128 for SpaR and minGRU. Because parameter counts differ across recurrent architectures at the same width, we include these larger widths so that all models cover similar ranges of parameter counts within each task. The effective MAC cost across all layers is reported in millions of MACs per second (M/s). This count excludes operations skipped through input sparsity in DuSpaR, SpaR, and Δ -GRU, or output sparsity in D-GRU.

4. RESULTS

Fig. 2 plots task performance against effective MAC cost for the three tasks. Curves toward the upper left indicate better performance with fewer computes. The SpaR achieves a better performance-computation trade-off than minGRU on SLU and SE, but still worse than GRU. Coupling two SpaR cells through feedback modulation to form DuSpaR improves computational efficiency. For the

DuSpaR-SpaR pairs marked by diamonds ($\blacklozenge$) in Fig. 2, DuSpaR reduces effective MAC cost by approximately 11.3 $\times$, 4.0 $\times$, and 3.1 $\times$ on KWS, SLU, and SE, respectively, at comparable task performance. Across all three tasks, DuSpaR achieves a substantially better performance-computation trade-off than GRU and D-GRU, as reflected by its curves lying further toward the upper left in Fig. 2. While Δ -GRU remains competitive for small models, DuSpaR holds a clear advantage for larger models.

Table 1 compares GRU, Δ -GRU, D-GRU, and DuSpaR at comparable parameter counts within each task. These configurations are marked by stars ($\star$) in Fig. 2. We use a larger layer width for DuSpaR so that its parameter count is close to, but slightly below, that of the other three models. DuSpaR combines the highest mean accuracy on KWS and SLU with the lowest effective MAC costs. On SLU, it achieves 93.12% accuracy, with an effective MAC cost that is lower by 68.4%, 41.6%, and 52.8% relative to GRU, Δ -GRU, and D-GRU, respectively. Its 38.3% occupancy is the lowest among the four models. On KWS, DuSpaR and Δ -GRU share the minimum cost of 3.21 MMACs/s with occupancies near 50%, but DuSpaR’s 93.72% accuracy is 0.6 percentage points higher. For SE, DuSpaR achieves the highest PESQ-WB score, with a small decrease in STOI and SI-SDR relative to GRU, but GRU requires approximately 2 $\times$ more MACs/s. Although Δ -GRU has a lower effective MAC cost for SE, DuSpaR provides higher PESQ-WB and STOI scores.

5. CONCLUSION

We introduce DuSpaR, a recurrent unit that combines coupled feed-forward and feedback states with ReLU-based vector sparsification to reduce computes for on-device inference of speech processing networks. Results show that DuSpaR achieves better performance-computation trade-offs on KWS, SLU, and SE tasks than the previous state-of-the-art models without the feedback recurrent cell. Future work includes experiments to further understand the role of the dual-state recurrence in improving the trade-off, extending the evaluation to larger models and tasks with longer temporal dependencies, e.g., long-form speech recognition and language modeling; and exploring applications in other modalities.

6. REFERENCES

- [1] Y. He et al., “Streaming End-to-End Speech Recognition for Mobile Devices,” in *ICASSP*, 2019, pp. 6381–6385.
- [2] I. Fedorov et al., “TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids,” in *Interspeech*, 2020, pp. 4054–4058.
- [3] J. Chung et al., “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling,” in *NIPS Workshop on Deep Learning*, 2014.
- [4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] X. Rong et al., “GTCRN: A Speech Enhancement Model Requiring Ultralow Computational Resources,” in *ICASSP*, 2024, pp. 971–975.
- [6] L. Cheng et al., “Dynamic Gated Recurrent Neural Network for Compute-Efficient Speech Enhancement,” in *Interspeech*, 2024, pp. 677–681.
- [7] A. Orvieto et al., “Resurrecting Recurrent Neural Networks for Long Sequences,” in *ICML*, 2023, pp. 26 670–26 698.
- [8] M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” in *NeurIPS*, vol. 37, 2024, pp. 107 547–107 603.
- [9] A. Gu et al., “Efficiently Modeling Long Sequences with Structured State Spaces,” in *ICLR*, 2022.
- [10] T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” in *ICML*, 2024, pp. 10 041–10 071.
- [11] A. Fasoli et al., “4-Bit Quantization of LSTM-Based Speech Recognition Models,” in *Interspeech*, 2021, pp. 2586–2590.
- [12] S. Cao et al., “Efficient and Effective Sparse LSTM on FPGA with Bank-Balanced Sparsity,” in *FPGA*, 2019, pp. 63–72.
- [13] D. Neil et al., “Delta Networks for Optimized Recurrent Network Computation,” in *ICML*, 2017, pp. 2584–2593.
- [14] A. Subramoney et al., “Efficient Recurrent Architectures Through Activity Sparsity and Sparse Back-Propagation Through Time,” in *ICLR*, 2023.
- [15] V. Campos et al., “Skip RNN: Learning to Skip State Updates in Recurrent Neural Networks,” in *ICLR*, 2018.
- [16] Y. Han et al., “Dynamic Neural Networks: A Survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7436–7456, 2022.
- [17] C. Gao et al., “EdgeDRNN: Recurrent Neural Network Accelerator for Edge Inference,” *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 10, no. 4, pp. 419–432, 2020.
- [18] S. Zhou et al., “An 8.62- μ W 75-dB DR_{SoC} Fully Integrated SoC for Spoken Language Understanding,” *IEEE J. Solid-State Circuits*, vol. 60, no. 11, pp. 4002–4017, 2025.
- [19] K. Han et al., “Deep Predictive Coding Network with Local Recurrent Processing for Object Recognition,” in *NeurIPS*, vol. 31, 2018.
- [20] D. J. Felleman and D. C. Van Essen, “Distributed hierarchical processing in the primate cerebral cortex,” *Cerebral cortex*, vol. 1, no. 1, pp. 1–47, 1991.
- [21] E. Martin and C. Cundy, “Parallelizing Linear Recurrent Neural Nets Over Sequence Length,” in *ICLR*, 2018.
- [22] L. Feng et al., “Were RNNs All We Needed?” *arXiv:2410.01201*, 2024.
- [23] C. Gao et al., “Spartus: A 9.4 TOP/s FPGA-Based LSTM Accelerator Exploiting Spatio-Temporal Sparsity,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 1, pp. 1098–1112, 2024.
- [24] Q. V. Le et al., “A Simple Way to Initialize Recurrent Networks of Rectified Linear Units,” *arXiv:1504.00941*, 2015.
- [25] M. Ravanelli et al., “Light Gated Recurrent Units for Speech Recognition,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 2, pp. 92–102, 2018.
- [26] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” *arXiv:1804.03209*, 2018.
- [27] L. Lugosch et al., “Speech Model Pre-Training for End-to-End Spoken Language Understanding,” in *Interspeech*, 2019, pp. 814–818.
- [28] A. Graves et al., “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *ICML*, 2006, pp. 369–376.
- [29] C. Valentini-Botinhao, “Noisy speech database for training speech enhancement algorithms and TTS models,” 2017.
- [30] C. Veaux et al., “The Voice Bank Corpus: Design, Collection and Data Analysis of a Large Regional Accent Speech Database,” in *Int. Conf. Oriental COCOSDA/CASLRE*, 2013, pp. 1–4.
- [31] J. Thiemann et al., “The Diverse Environments Multi-Channel Acoustic Noise Database (DEMAND): A Database of Multichannel Environmental Noise Recordings,” *Proceedings of Meetings on Acoustics*, vol. 19, no. 1, 2013, Art. no. 035081.
- [32] L. Cheng et al., “Modulating State Space Model with Slow-Fast Framework for Compute-Efficient Ultra Low-Latency Speech Enhancement,” in *ICASSP*, 2025, pp. 1–5.
- [33] J. M. Martin-Donas et al., “A Deep Learning Loss Function Based on the Perceptual Evaluation of the Speech Quality,” *IEEE Signal Process. Lett.*, vol. 25, no. 11, pp. 1680–1684, 2018.
- [34] H. Zhang et al., “Training Supervised Speech Separation System to Improve STOI and PESQ Directly,” in *ICASSP*, 2018, pp. 5374–5378.
- [35] A. W. Rix et al., “Perceptual Evaluation of Speech Quality (PESQ) - A New Method for Speech Quality Assessment of Telephone Networks and Codecs,” in *ICASSP*, 2001, pp. 749–752.
- [36] C. H. Taal et al., “An Algorithm for Intelligibility Prediction of Time-Frequency Weighted Noisy Speech,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 7, pp. 2125–2136, 2011.
- [37] J. Le Roux et al., “SDR – Half-Baked or Well Done?” in *ICASSP*, 2019, pp. 626–630.